

TASK 1: AMDAHL'S LAW & PHYSICAL SILICON SATURATION

Worker Cores (p)	Measured Time T(p)	Observed Speedup S(p)	Ideal Linear Speedup	Parallel Efficiency
1	2.4443 s	1.02×	1.00×	102.0%
2	1.7011 s	1.47×	2.00×	73.5%
4	1.4265 s	1.75×	4.00×	43.75%
8	1.4522 s	1.72×	8.00×	21.5%
12	1.6045 s	1.55×	12.00×	12.92%
16	1.7265 s	1.44×	16.00×	9.0%

[Q1.1] Identify the exact inflection point (core count p^*) where your observed speedup curve flattens or degrades. Correlate this directly to your machine's physical hardware topology (distinguish between physical Performance cores, Efficiency cores, and logical SMT hyperthreads).

The observed speedup increases from 1.02× with 1 worker to 1.47× with 2 workers and reaches the maximum of 1.75× at 4 workers. After 4 workers, the speedup starts to decrease: 1.72× at 8 workers, 1.55× at 12 workers, and 1.44× at 16 workers.

Therefore, the observed inflection point is approximately $p^* = 4$ workers. My machine reports 4 logical CPU cores, so the best performance in this experiment occurs around the available logical-core capacity. Using more worker processes than the available logical cores causes additional process scheduling and resource contention, so performance decreases instead of continuing to scale linearly.

[Q1.2] Using your empirical measurements for T(1) and T(4), calculate the exact value of P (the parallel fraction) of this workload on your silicon. Show the complete algebraic derivation: $S(4) = T(1)/T(4) = 1 / [(1 - P) + P/4]$.

Using the measured baseline $T(1) = 2.4934$ s and the measured $T(4) = 1.4265$ s:

$$S(4) = T(1) / T(4)$$

$$S(4) = 2.4934 / 1.4265 \approx 1.7482$$

$$\text{According to Amdahl's Law: } S(4) = 1 / [(1 - P) + P/4]$$

Therefore:

$$1/1.7482 = 1 - P + P/4$$

$$0.5720 = 1 - 3P/4$$

$$3P/4 = 0.4280$$

$$P \approx 0.571$$

Therefore, the measured parallel fraction is approximately 57.1%. The remaining approximately 42.9% represents the non-parallel or serial fraction of the workload.

[Q1.3] Compute the Asymptotic Speedup Limit $S_{\max} = \lim_{p \rightarrow \infty} S(p) = 1 / (1 - P)$ using your calculated P from Q1.2. Explain what this limit implies if your department purchased a 128-core server node to run this specific workload.

Using the parallel fraction calculated in Q1.2, $P \approx 0.5707$:

$$S_{\max} = 1 / (1 - P)$$

$$S_{\max} = 1 / (1 - 0.5707)$$

$$S_{\max} \approx 2.33\times$$

Therefore, the theoretical maximum speedup for this workload is approximately 2.33 $\times$, even with an unlimited number of processors.

If the department purchased a 128-core server node, this particular workload would not scale to 128 $\times$ performance. A large part of the workload remains serial, and additional cores would eventually provide little or no benefit. In the real experiment, the performance also decreases after 4 workers because of process overhead, scheduling, and CPU resource contention.

TASK 2: CACHE COHERENCE & THE FALSE SHARING TRAP

Memory Layout	Trial 1	Trial 2	Trial 3	Median Time	Slowdown
Adjacent Indices (False Sharing Active)	0.8198 s	0.7952 s	0.7890 s	0.7952 s	0.996$\times$
Padded Indices (Cache-Aligned)	0.8068 s	0.7786 s	0.7981 s	0.7981 s	0.996$\times$

[Q2.1] Calculate the exact memory footprint of a 64-bit integer pointer/reference in CPython. Mathematically prove why an index stride of 16 guarantees that adjacent threads never access memory within the same 64-byte L1 cache line.

On a 64-bit CPython implementation, a Python list stores 8-byte references (pointers) to Python objects. The Python integer object itself is larger than 8 bytes, but the list entry/reference occupies 8 bytes. The padded worker uses an index stride of 16. Therefore, the distance between two adjacent selected list references is:

$$16 \times 8 \text{ bytes} = 128 \text{ bytes.}$$

A cache line is 64 bytes:

$$128 \text{ bytes} > 64 \text{ bytes.}$$

Therefore, two adjacent padded indices are separated by 128 bytes and cannot be located within the same 64-byte cache line. This guarantees that the selected list-reference locations are at least two cache-line widths apart.

[Q2.2] Trace the MESI state transitions for the shared cache line containing array elements. When Core 0 executes an increment on index 0 and Core 1 subsequently executes an increment on index 1, describe the bus invalidation messages and state shifts (Shared -> Modified -> Invalid).

When two cores modify different variables located on the same cache line, the MESI coherence protocol must maintain a single consistent copy of that cache line.

Initially, the cache line can be in the Shared (S) state in both cores. When Core 0 writes to index 0, it needs exclusive ownership of the cache line, so the line changes from Shared to Modified (M) in Core 0. The coherence system sends an invalidation request to other cores holding the same line.

When Core 1 subsequently writes to index 1, its copy of the line has become Invalid (I). Core 1 must obtain ownership of the cache line before modifying it. This causes another invalidation of Core 0's copy.

The resulting pattern is effectively:

Core 0: Shared → Modified

Core 1: Shared → Invalid

Core 1: Invalid → Modified

Core 0: Modified → Invalid

Repeated writes cause the cache line to move between cores, creating cache-line ping-pong and additional coherence traffic. This is the fundamental mechanism behind false sharing.

[Q2.3] In compiled systems programming, manual array padding wastes memory. Describe two industry-standard compile-time or VM-level mechanisms used to eliminate false sharing (e.g., C++11 'alignas(64)', Java '@Contended', or Go struct field padding).

Two industry-standard mechanisms for reducing false sharing are:

1. C++11 alignas(64) — this allows a programmer to explicitly align frequently modified variables to a 64-byte cache-line boundary. Additional padding can be used to ensure that independent variables occupy separate cache lines.
2. Java @Contended — this JVM-level annotation can separate frequently modified fields by adding internal padding. It is designed to prevent unrelated fields accessed by different threads from sharing the same cache line.

Both techniques reduce cache-coherence traffic by ensuring that independent frequently written variables are separated in memory.

TASK 3: THE SYNCHRONIZATION TAX & LOCKLESS REDESIGN

Counter Architecture	Target Sum	Actual Empirical Sum	Data Corruption Defect Count	Execution Time [s]	Contention Overhead
UnsafeCounter	2,000,000	2,000,000	0	0.1325	Baseline
LockedCounter	2,000,000	2,000,000	0	0.4288	3.24×

[Q3.1] Analyze your Unsafe Accumulator output. What was your exact corrupted count? Explain the low-level readmodify-write machine code sequence (LOAD, ADD, STORE) that caused lost updates between overlapping thread executions.

In my empirical run, the UnsafeCounter produced the correct value of 2,000,000, so the observed data corruption was 0. The LockedCounter also produced 2,000,000.

In a true race condition, the operation `self.val += 1` can be viewed as three steps: LOAD the current value, ADD 1, and STORE the new value. If two threads read the same value before either stores its result, one update can overwrite the other. For example, if both threads load 100, both calculate 101, and both store 101, two increments produce only one effective increment. This is called a lost update.

In my Python 3.13 run, this corruption did not appear, most likely because CPython's GIL limits simultaneous execution of Python bytecode. Therefore, the theoretical LOAD-ADD-STORE race is important, but it was not empirically observed in this particular run.

[Q3.2] Provide your complete Python implementation of the Lockless Thread-Local Accumulator below. Record your measured execution time and calculate the speedup factor achieved over LockedCounter.

The lockless redesign uses thread-local accumulation buffers. Each thread performs its operations on its own private variable, so no mutex is required during the main computation. After all threads finish, the partial results are combined using a final reduction.

Empirical result:

- Target: 2,000,000
- Actual result: 2,000,000
- Data corruption: 0
- Lockless execution time: 0.0546 s
- LockedCounter execution time: 0.4288 s
- Speedup: 7.86×

The result is exactly 2,000,000, so the implementation is correct. The measured speedup of 7.86× is greater than the required 2.0× target.

[Q3.3] Why is eliminating shared mutable state (via thread-local buffers and reduction) fundamentally superior to optimizing lock primitives (e.g., adaptive spinlocks or read-write locks) in high-throughput concurrent systems?

Thread-local accumulation is better because it removes shared mutable state from the main computation. Each thread works independently on its own private data, so there is no need to acquire and release a lock for every operation.

Optimizing lock primitives, such as adaptive spinlocks or read-write locks, can reduce some synchronization overhead, but the lock is still a serialization point when many threads access the same data. Thread-local buffers avoid this problem almost completely and reduce contention, cache-coherence traffic, and waiting between threads.

The final reduction is performed only after the parallel work is complete. This design therefore provides both correctness and better performance for high-throughput workloads.

Architecture	Actual Sum	Defects	Time	Overhead / Speedup
UnsafeCounter	2,000,000	0	0.1325 s	Baseline
LockedCounter	2,000,000	0	0.4288 s	3.24× slower
Lockless Thread-Local	2,000,000	0	0.0546 s	7.86× faster than Locked